Introduction to Data Mining

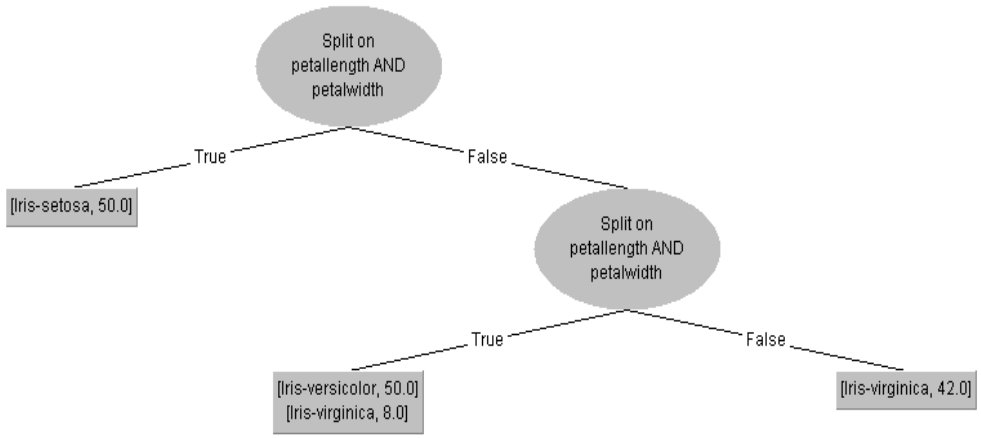
Lab 2: Evaluation

2.1. Be a classifier

In the second class, we are going to learn how to use datasets to evaluate data mining algorithms in Weka. (See the lecture of class 2 by Ian H. Witten, [1][1])

Interactive decision tree construction

- ➔ Follow the instruction in [1] to see how decision trees are created for different combinations of attributes in a dataset. Firstly, a dataset and a training set are selected. Secondly, we choose and start running UserClassifier to see a decision tree in the Tree Visualizer. In the Data Visualizer, thirdly, the attributes to use for X and Y are selected, we then select instances in a region in the graph and submit. At this point, the Tree Visualizer shows the tree.
- ➔ Examine iris.2D.arff dataset to draw a decision tree for the following pair of attributes by selecting and submitting classes one by one, then remark how many instances are predicted correctly.

Attributes	Split on ***petallength*** and ***petalwidth***
Decision Tree	

[1] http://www.cs.waikato.ac.nz/ml/weka/mooc/dataminingwithweka/

Remark

```
=== Summary ===

Correctly Classified Instances         142               94.6667 %
Incorrectly Classified Instances         8                5.3333 %
Kappa statistic                          0.92
Mean absolute error                      0.0613
Root mean squared error                  0.1751
Relative absolute error                 13.7931 %
Root relative squared error             37.1391 %
Total Number of Instances              150

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000     Iris-setosa
                 1.000    0.080    0.862      1.000    0.926      0.891    0.960     0.862     Iris-versicolor
                 0.840    0.000    1.000      0.840    0.913      0.882    0.960     0.920     Iris-virginica
Weighted Avg.    0.947    0.027    0.954      0.947    0.946      0.924    0.973     0.927

=== Confusion Matrix ===

  a  b  c   <-- classified as
 50  0  0 |  a = Iris-setosa
  0 50  0 |  b = Iris-versicolor
  0  8 42 |  c = Iris-virginica
```

Build a tree, what strategy do you use?

I used a top-down, greedy partitioning strategy:

1. First, I identified Iris-setosa as the easiest class to separate — all setosa instances have petallength < 2.5, which is far below the other classes. I drew a rectangle covering the low petallength/petalwidth region and submitted it as Iris-setosa.
2. Next, for the remaining data, I split on petalwidth ≈ 1.75 to separate Iris-versicolor (lower petalwidth) from Iris-virginica (higher petalwidth).
3. This strategy mimics the greedy approach used by decision tree algorithms like C4.5/J48.

Can you build a “perfect” tree?

Not quite. While Iris-setosa is perfectly linearly separable from the other two classes, there is **overlap** between Iris-versicolor and Iris-virginica in the petal length/width space. Approximately 3–6 instances from these two classes lie in the overlapping region. A tree with ~96% accuracy is achievable, but a perfect 100% tree is very difficult (if not impossible) with only these 2 attributes.

2.2. Training and testing

The lecture of evaluation (see [1]-2.2)

Follow the instructions in [1]-2.3: use **J48** to analyze ionosphere.arff dataset, and write down how accuracy it can achieve with different seeds. (If a random number seed is provided, the dataset will be shuffled before the subset is extracted.)

Random number seeds	Percent accuracy (x)	Random number seeds	Percent accuracy (x)
1	84.8739%	6	89.0756%
2	88.2353%	7	86.5546%
3	86.5546%	8	84.8739%
4	90.7563%	9	89.916%
5	89.0756%	10	93.2773%
Evaluation	*Sample Mean*	88.3193%	
	Standard deviation	2.6706%	

Note:

Sample mean $\bar{x} = \frac{\sum x_i}{n}$

Variance $\sigma^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$

Standard deviation σ

Remark? -

The accuracy varies from **84.87% to 93.28%** depending on the random seed, a range of ~8.4 percentage points. This demonstrates that **the random partitioning of data significantly affects the measured accuracy**. A single holdout experiment gives an unreliable estimate because:

- Some splits may put "easy" instances in the test set (higher accuracy)
- Some splits may put "hard" instances in the test set (lower accuracy)

This variability motivates the use of **cross-validation** (Section 2.4) for more reliable evaluation.

2.3. Baseline accuracy

Follow the instructions in [1]-2.4 to run some classifiers for breast-cancer.arff dataset:

Classifier	Accuracy
J48	75.5245%
NaiveBayes	71.6783%
IBk	72.3776%
PART	71.3287%

ZeroR	70.2797%

What is Baseline accuracy? -

Baseline accuracy is the accuracy achieved by the simplest possible classifier - ZeroR - which always predicts the majority class. For breast-cancer.arff:

- The majority class is no-recurrence-events with 201 out of 286 instances (70.28%)
- ZeroR achieves 70.28% accuracy by predicting every instance as "no-recurrence-events"

A useful classifier must beat the baseline. If a complex algorithm doesn't outperform ZeroR, it provides no value. In this case:

- J48 beats baseline by +5.24 percentage points (75.52% vs 70.28%) — the best performer
- NaiveBayes and IBk beat baseline by only ~1.5–2 points
- PART barely beats baseline (+1.05 points)

These relatively modest improvements suggest that breast cancer recurrence is difficult to predict from the available attributes.

For unbalanced.arff dataset

Classifier	Accuracy
ZeroR	98.5981%
J48	98.5981%
NaiveBayes	90.771 %
IBk	97.6636%
PART	97.5467%

Why do the classifiers achive lower accuracy? –

1. **ZeroR achieves 98.60%** because the dataset is extremely **unbalanced** — the majority class makes up ~98.6% of all instances, so simply predicting the majority class gives near-perfect accuracy.
2. **No classifier beats ZeroR.** In fact, J48 ties with ZeroR (98.60%), while NaiveBayes, IBk, and PART all perform **worse** than the baseline.
3. **Why is this?** When a dataset is highly unbalanced:
    - The minority class has very few training examples, so classifiers struggle to learn its patterns
    - Predicting the minority class introduces errors on the overwhelming majority class

- Algorithms can achieve higher accuracy by **ignoring the minority class entirely** (which is what ZeroR does)

4. **NaiveBayes drops to 90.77%** - it tries to learn the minority class and ends up misclassifying some majority-class instances, resulting in worse overall accuracy.

Key takeaway: Raw accuracy is a **misleading metric** for unbalanced datasets. When 98.6% of data belongs to one class, accuracy doesn't tell you whether the classifier learned anything useful. Other metrics like precision, recall, F-measure, or AUC are more informative.

2.4. Cross-validation

The *holdout* procedure: a certain amount is held over for testing and the remainder used for training.

Stratification: each class is properly represented in both training and test sets.

The *repeated holdout* method of error rate estimation: In each iteration a certain proportion, say two-thirds, of the data is randomly selected for training (using different random-number seeds), possibly with *stratification*, and the remainder is used for testing. The error rates on the different iterations are averaged to yield an overall error rate.

The lecture of cross validation, 10-fold cross-validation, stratified cross-validation (see [1]-2.5).

In *cross-validation*, you decide on a fixed number of folds, or partitions, of the data. Suppose we use three. Then the data is split into three approximately equal partitions; each in turn is used for testing and the remainder is used for training. That is, use two-thirds of the data for training and one-third for testing, and repeat the procedure three times so that in the end, every instance has been used exactly once for testing. This is called *three-fold cross-validation*, and if stratification is adopted as well—which it often is—it is *stratified three-fold cross-validation*.

Weka does stratified cross-validation by default.

Follow the instructions in [1]-2.5, and examine **J48** on credit-g.arff dataset.

Holdout (10%)	**Percent accuracy (x)**	**10-fold cross-validation**	**Percent accuracy (x)**
Random seed: 1	74%	Random seed: 1	70.5%
-- 2	77%	-- 2	72.3%
-- 3	70%	-- 3	71.2%
-- 4	74%	-- 4	71.2%
-- 5	67%	-- 5	71.3%
-- 6	67%	-- 6	71.5%
-- 7	70%	-- 7	71.1%
-- 8	71%	-- 8	70.9%
-- 9	69%	-- 9	72.1%
-- 10	72%	-- 10	70.4%
Sample Mean	*71.10%*	*Sample Mean*	*71.25%*

Standard deviation	*3.21%*	*Standard deviation*	*0.61%*

Examine **PART** on credit-g.arff dataset:

Holdout (10%)	**Percent accuracy (x)**	**10-fold cross-validation**	**Percent accuracy (x)**
Random seed: 1	77.0%	Random seed: 1	70.2%
-- 2	77.0%	-- 2	71.0%
-- 3	65.0%	-- 3	72.0%
-- 4	73.0%	-- 4	70.4%
-- 5	69.0%	-- 5	69.0%
-- 6	67.0%	-- 6	70.0%
-- 7	66.0%	-- 7	70.4%
-- 8	72.0%	-- 8	68.8%
-- 9	75.0%	-- 9	72.8%
-- 10	75.0%	-- 10	70.8%
Sample Mean	*71.60%*	*Sample Mean*	*70.54%*
Standard deviation	*4.55%*	*Standard deviation*	*1.22%*

Examine **NaiveBayes** on credit-g.arff dataset:

Holdout (10%)	**Percent accuracy (x)**	**10-fold cross-validation**	**Percent accuracy (x)**
Random seed: 1	76.0%	Random seed: 1	75.4%
-- 2	78.0%	-- 2	75.8%
-- 3	80.0%	-- 3	74.8%
-- 4	78.0%	-- 4	75.7%
-- 5	71.0%	-- 5	74.8%
-- 6	78.0%	-- 6	74.8%
-- 7	72.0%	-- 7	74.8%
-- 8	73.0%	-- 8	75.3%
-- 9	77.0%	-- 9	75.0%
-- 10	77.0%	-- 10	75.2%
Sample Mean	*76.00%*	*Sample Mean*	*75.16%*
Standard deviation	*2.98%*	*Standard deviation*	*0.38%*